

Pramo Jewels

Role-Based Access Control (RBAC)

Document ID: PJ-RBAC-001

Version: 1.0

Status: Approved for Development

Field	Value
Project	Pramo Jewels Premium E-Commerce Platform
Purpose	Define roles, permissions, access rules and RBAC implementation standards
Audience	Security Architects, Backend Developers, DevOps, QA
Version	1.0

Permission Matrix

Permission Area	Customer	Support	Inventory	Admin	Super Admin
View Products	✓	✓	✓	✓	✓
Manage Profile	✓			✓	✓
Manage Orders	Own	Assist		✓	✓
Manage Products			✓	✓	✓
Manage Inventory			✓	✓	✓
Manage Coupons				✓	✓
Manage Customers		View		✓	✓
Reports		View	View	✓	✓
System Settings					✓
Roles & Permissions					✓
Audit Logs		View		✓	✓

RBAC Principles

Apply least privilege, separation of duties and deny-by-default. Users receive only the permissions required for their responsibilities.

Role Definitions

Customer, Support, Inventory Manager, Admin and Super Admin are the default roles. Additional custom roles may be introduced through configuration rather than code changes.

Authorization Flow

User authenticates → Role resolved → Permissions loaded → Middleware & Policy checks → Resource access granted or denied.

Implementation

Use Laravel Gates, Policies, middleware and permission caching. Centralize permission definitions and avoid hard-coded authorization logic in controllers.

Permission Management

Only Super Admins may create roles or modify permissions. All permission changes require audit logging.

Audit & Compliance

Record role assignments, permission changes, denied access attempts and privileged actions with timestamps and actor identity.

Security Controls

Review role assignments periodically, disable dormant privileged accounts, require strong authentication for administrators and support future MFA.

Acceptance Criteria

Every protected endpoint enforces RBAC, unauthorized access returns HTTP 403, and permission changes are fully traceable.